# THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

## DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms. Jarina Banu H Reg. No: 20524UCSC016, a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course Incharge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

# CONTENTS


| S.NO. | DATE | TITLE | PAGE NO | SIGNATURE |
|---|---|---|---|---|
| **1.** | 15-06-2026 | DDL Commands | 1 | |
| **2.** | 15-06-2026 | DML Commands | 4 | |
| **3.** | 15-06-2026 | DQL Command | 6 | |
| **4.** | 15-06-2026 | TCL Command | 7 | |
| **5.** | 19-06-2026 | Constraints | 10 | |
| **6.** | 19-06-2026 | Joins | 12 | |
| **7.** | 19-06-2026 | Aggregate Functions | 16 | |
| **8.** | 19-06-2026 | View Creations | 18 | |
| **9.** | 19-06-2026 | Set Operators | 19 | |
| **10.** | 23-06-2026 | String Function | 21 | |
| **11.** | 23-06-2026 | Numeric Function | 23 | |
| **12.** | 23-06-2026 | Nested Subquery | 24 | |
| **13.** | 23-06-2026 | PL/SQL - Fibonacci Series | 27 | |
| **14.** | 30-06-2026 | PL/SQL - Factorial | 28 | |
| **15.** | 30-06-2026 | PL/SQL - String Reverse | 29 | |
| **16.** | 30-06-2026 | PL/SQL - Sum of Series | 30 | |

| | | | | |
|---|---|---|---|---|
| **17.** | 30-06-2026 | PL/SQL - Trigger | 31 | |
| **18.** | 30-06-2026 | PL/SQL - Student Mark and Grade Calculation | 33 | |
| **19.** | 30-06-2026 | PL/SQL - Employee Salary Calculation | 35 | |
| **20.** | 02-07-2026 | PL/SQL - Electricity Bill Calculation | 36 | |
| **21.** | 02-07-2026 | PL/SQL - Bank Deposit and Interest Calculation | 37 | |
| **22.** | 08-07-2026 | PL/SQL - ATM Withdrawal | 38 | |
| **23.** | 08-07-2026 | PL/SQL - Product Discount Calculation | 39 | |
| **24.** | 08-07-2026 | PL/SQL - Employee Bonus Calculation | 41 | |
| **25.** | 10-07-2026 | PL/SQL - Student Attendance Eligibility | 42 | |
| **26.** | 10-07-2026 | PL/SQL - Armstrong Number | 43 | |
| **27.** | 10-07-2026 | PL/SQL - Cursor | 45 | |
| **28.** | 16-07-2026 | PL/SQL - Check Even or Odd Number | 48 | |
| **29.** | 16-07-2026 | PL/SQL - Palindrome Checker | 49 | |
| **30.** | 16-07-2026 | PL/SQL - Prime Number | 51 | |
| **31.** | 05-08-2026 | PL/SQL - GCD | 52 | |
| **32.** | 05-08-2026 | PL/SQL - Operators | 53 | |

## 1. DDL COMMANDS

**Table Creation:**

SQL > create table flight_booking (cid number(10), cname varchar(20), ctp number(10));

Results Explain Describe Saved SQL History

```
Table created.
```

0.10 seconds

**Table Description:**

SQL> desc flight_booking;

Results Explain Describe Saved SQL History

Object Type TABLE Object FLIGHT_BOOKING

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| FLIGHT_BOOKING | CID | Number | - | 10 | 0 | - | ✓ | - | - |
| | CNAME | Varchar2 | 20 | - | - | - | ✓ | - | - |
| | CTP | Number | - | 10 | 0 | - | ✓ | - | - |

1 - 3

**Table Alteration:**

**Method 1(Add)**

SQL> alter table flight_booking add cphn number(10);

Results Explain Describe Saved SQL History

```
Table altered.
```

0.05 seconds

SQL> select*from flight_booking;

Results Explain Describe Saved SQL History

| CID | CNAME | CTP | CPHN |
|---|---|---|---|
| 111 | Savitha | 30000 | - |
| 112 | Priya | 40000 | - |
| 113 | Raji | 20000 | - |
| 114 | Bavana | 50000 | - |

4 rows returned in 0.05 seconds

**Method 2(Drop)**

SQL> alter table student drop column class;

Results Explain Describe Saved SQL History

```
Table dropped.
```

0.27 seconds

SQL> select*from flight_booking;

Results Explain Describe Saved SQL History

| CID | CNAME | CTP |
|---|---|---|
| 111 | Savitha | 30000 |
| 112 | Priya | 40000 |
| 113 | Raji | 20000 |
| 114 | Bavana | 50000 |

4 rows returned in 0.01 seconds

**Rename Table:**

SQL> rename flight_booking to f_b;

Results Explain Describe Saved SQL History

```
Statement processed.
```

0.05 seconds

**Truncate Table:**

SQL> truncate table f_b;




```
Table truncated.
```

0.26 seconds

**Drop Table:**

SQL> drop table f_b;




```
Table dropped.
```

0.36 seconds

## 2. DML COMMANDS

**Table Insertion:**

SQL> insert into flight_booking values(111, 'Savitha', 30000);

1 row(s) inserted.

SQL> insert into flight_booking values(112, 'Priya', 40000);

1 row(s) inserted.

SQL> insert into flight_booking values(113, 'Raji', 20000);

1 row(s) inserted.

SQL> insert into flight_booking values(114, 'Bavana', 50000);

1 row(s) inserted.

**Table updating:**

SQL> update f_b set cname='Varsha' where cid=111;

Results Explain Describe Saved SQL History

1 row(s) updated.

0.01 seconds

SQL> select*from f_b;

Results Explain Describe Saved SQL History

| CID | CNAME | CTP |
|---|---|---|
| 111 | Varsha | 30000 |
| 112 | Priya | 40000 |
| 113 | Raji | 20000 |
| 114 | Bavana | 50000 |

4 rows returned in 0.00 seconds

**Table Deletion:**

SQL>delete from f_b where cid=111;



## 3. DQL COMMAND

**Table View:**

SQL> select*from f_b;

Results Explain Describe Saved SQL History

| CID | CNAME | CTP |
|---|---|---|
| 112 | Priya | 40000 |
| 113 | Raji | 20000 |
| 114 | Bavana | 50000 |

3 rows returned in 0.00 seconds

## 4. TCL COMMAND

**ROLLBACK**

SQL> delete from f_b where cid=111;

Results Explain Describe Saved SQL History

```
1 row(s) deleted.
```

0.00 seconds

SQL> select*from f_b;

Results Explain Describe Saved SQL History

| CID | CNAME | CTP |
|---|---|---|
| 112 | Priya | 40000 |
| 113 | Raji | 20000 |
| 114 | Bavana | 50000 |

3 rows returned in 0.00 seconds

SQL> rollback;

Results Explain Describe Saved SQL History

```
Statement processed.
```

0.00 seconds

SQL> select*from f_b;

Results Explain Describe Saved SQL History

| CID | CNAME | CTP |
|---|---|---|
| 111 | Savitha | 30000 |
| 112 | Priya | 40000 |
| 113 | Raji | 20000 |
| 114 | Bavana | 50000 |

4 rows returned in 0.00 seconds

**SAVEPOINT**

```
SQL> begin
        savepoint p;
     end;
```

Results Explain Describe Saved SQL History

```
Statement processed.
```

0.00 seconds

```
SQL> insert into f_b values(115,'Gaja',80000);
```

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

```
SQL> select*from f_b;
```

Results Explain Describe Saved SQL History

| CID | CNAME | CTP |
|---|---|---|
| 111 | Savitha | 30000 |
| 112 | Priya | 40000 |
| 113 | Raji | 20000 |
| 114 | Bavana | 50000 |
| 115 | Gaja | 80000 |

5 rows returned in 0.00 seconds

```
SQL> rollback to p;
SQL> select*from f_b;
```

Results Explain Describe Saved SQL History

| CID | CNAME | CTP |
|---|---|---|
| 111 | Savitha | 30000 |
| 112 | Priya | 40000 |
| 113 | Raji | 20000 |
| 114 | Bavana | 50000 |

4 rows returned in 0.00 seconds

**COMMIT**

SQL> commit;

Results Explain Describe Saved SQL History

```
Statement processed.
```

0.00 seconds

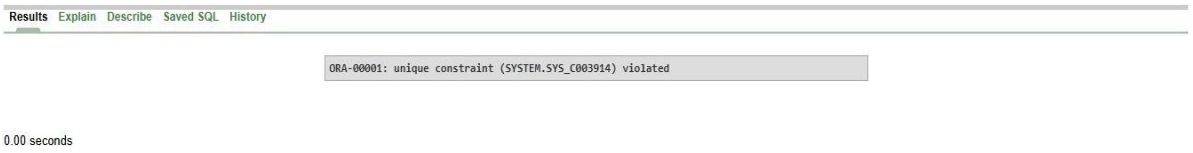
## 5. CONSTRAINTS

**UNIQUE:**

SQL> create table food(fid number(10), fname varchar(20) unique);

SQL> insert into food values(01,'dosa');

SQL> insert into food values(02,'dosa');




**PRIMARY KEY:**

SQL> alter table food add constraint tk primary key(fid);

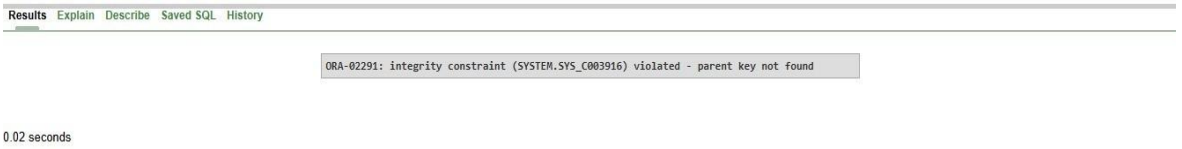
SQL> insert into food values(01,'puri');




**FOREIGN KEY:**

SQL> create table hotel(fid number(10), fname varchar(20), foreign key(fid) references food(fid));

SQL> insert into hotel values(03,'pongal');



**CHECK:**

SQL> alter table food add constraint ck check(fid<20);

SQL> insert into food values(21,'idly');



# 6. JOINS

**TABLE - 1**

SQL> create table employee(eno number(10), empname varchar(20), city varchar(20));

Table Created

**TABLE INSERTION**

SQL> insert into employee values(1001, 'Savitha', 'Pondy')

1 row(s) inserted.

SQL> insert into employee values(1002, 'Priya', 'Villupuram')

1 row(s) inserted.

SQL> insert into employee values(1003, 'Raji', 'Madukkarai')

1 row(s) inserted.

**VIEW TABLE**

SQL> select * from employee

Results Explain Describe Saved SQL History

| ENO | EMPNAME | CITY |
|---|---|---|
| 1001 | Savitha | Pondy |
| 1002 | Priya | Villupuram |
| 1003 | Raji | Madukkarai |

3 rows returned in 0.02 seconds CSV Export

**TABLE - 2**

SQL> create table emp_sal(emp_name varchar(20),dept varchar(20),salary number(10))

**TABLE INSERTION**

SQL> insert into emp_sal values('Savitha','CS',60000)

1 row(s) inserted.

SQL> insert into emp_sal values('Bavana','IT',70000)

1 row(s) inserted.

SQL> insert into emp_sal values('Raji','BCA',80000)

1 row(s) inserted.

**VIEW TABLE**

SQL> select * from emp_sal

Results Explain Describe Saved SQL History

| EMP_NAME | DEPT | SALARY |
|---|---|---|
| Saviha | CS | 60000 |
| Bavana | IT | 70000 |
| Raji | BCA | 80000 |

3 rows returned in 0.20 seconds CSV Export

**INNER JOIN**

SQL> select * from employee inner join emp_sal on employee.empname = emp_sal.emp_name;

**OUTPUT**

Results Explain Describe Saved SQL History

| ENO | EMPNAME | CITY | EMP_NAME | DEPT | SALARY |
|---|---|---|---|---|---|
| 1001 | Savitha | Pondy | Savitha | CS | 60000 |
| 1003 | Raji | Madukkarai | Raji | BCA | 80000 |

2 rows returned in 0.00 seconds CSV Export

**LEFT OUTER JOIN**

SQL> select e.empname, s.salary from employee e left outer join emp_sal s on employee.empname=emp_sal.emp_name;

**OUTPUT**

Results Explain Describe Saved SQL History

| EMPNAME | SALARY |
|---|---|
| Savitha | 60000 |
| Raji | 80000 |
| Priya | - |

3 rows returned in 0.00 seconds CSV Export

**RIGHT OUTER JOIN**

SQL> select e.empname, e.city, s.salary from employee e right outer join emp_sal s on e.empname=s.emp_name;

**OUTPUT**

Results Explain Describe Saved SQL History

| EMPNAME | CITY | SALARY |
|---|---|---|
| Savitha | Pondy | 60000 |
| Raji | Madukkarai | 80000 |
| - | - | 70000 |

3 rows returned in 0.00 seconds CSV Export

**NATURAL FULL JOIN**

SQL> select * from employee natural full join emp_sal;

**OUTPUT**

Results Explain Describe Saved SQL History

| ENO | EMPNAME | CITY | EMP_NAME | DEPT | SALARY |
|---|---|---|---|---|---|
| 1001 | Savitha | Pondy | Savitha | CS | 60000 |
| 1001 | Savitha | Pondy | Bavana | IT | 70000 |
| 1001 | Savitha | Pondy | Raji | BCA | 80000 |
| 1002 | Priya | Villupuram | Savitha | CS | 60000 |
| 1002 | Priya | Villupuram | Bavana | IT | 70000 |
| 1002 | Priya | Villupuram | Raji | BCA | 80000 |
| 1003 | Raji | Madukkarai | Savitha | CS | 60000 |
| 1003 | Raji | Madukkarai | Bavana | IT | 70000 |
| 1003 | Raji | Madukkarai | Raji | BCA | 80000 |

9 rows returned in 0.04 seconds CSV Export

# 7. AGGREGATE FUNCTIONS

**SUM**

SQL> select sum(salary) "Total" from emp_sal;

**OUTPUT**




**MAX**

SQL> select max (salary)"Maximum Salary" from emp_sal;

**OUTPUT**




**MIN**

SQL> select min (salary)"Minimum Salary" from emp_sal;

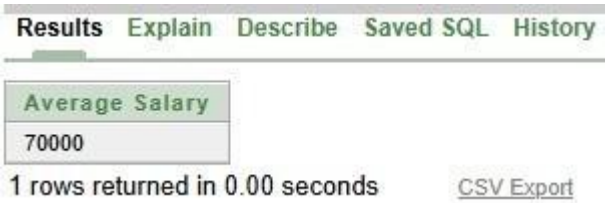
**OUTPUT**




**AVERAGE**

SQL> select avg (salary)"Average Salary" from emp_sal;

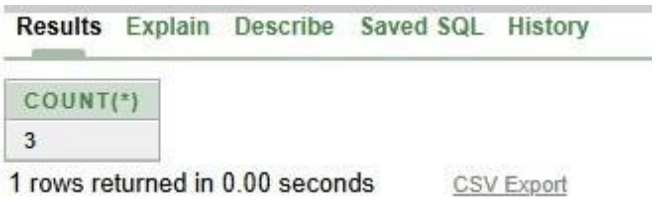
**OUTPUT**




**COUNT**

SQL> select count(*) from emp_sal;

**OUTPUT**



# 8. VIEW CREATIONS

SQL> create view employee_view as select eno, empname from employee;

**OUTPUT**

Results Explain Describe Saved SQL History

```
View created.
```

0.07 seconds

SQL> select * from employee_view;

**OUTPUT**

Results Explain Describe Saved SQL History

| ENO | EMPNAME |
|---|---|
| 1001 | Savitha |
| 1002 | Priya |
| 1003 | Raji |

3 rows returned in 0.02 seconds CSV Export

# 9. SET OPERATORS

**UNION**

SQL> select * from employee UNION select * from emp_sal

**OUTPUT**

Results Explain Describe Saved SQL History

| EMPNAME | CITY |
|---|---|
| Bavana | IT |
| Priya | Villupuram |
| Raji | BCA |
| Raji | Madukkarai |
| Savitha | CS |
| Savitha | Pondy |

6 rows returned in 0.00 seconds CSV Export

**UNION ALL**

SQL> select * from employee UNION ALL select * from emp_sal

**OUTPUT**

Results Explain Describe Saved SQL History

| EMPNAME | CITY |
|---|---|
| Bavana | IT |
| Priya | Villupuram |
| Raji | BCA |
| Raji | Madukkarai |
| Savitha | CS |
| Savitha | Pondy |

6 rows returned in 0.00 seconds CSV Export

**INTERSECT**

SQL> select empname from employee INTERSECT select emp_name from emp_sal

**OUTPUT**

Results Explain Describe Saved SQL History

| EMPNAME |
| --- |
| Raji |
| Savitha |

2 rows returned in 0.02 seconds CSV Export

**MINUS**

SQL> select * from employee MINUS select * from emp_sal

**OUTPUT**

Results Explain Describe Saved SQL History

| EMPNAME | CITY |
| --- | --- |
| Priya | Villupuram |
| Raji | Madukkarai |
| Savitha | Pondy |

3 rows returned in 0.02 seconds CSV Export

# 10. STRING FUNCTION

**REPLACE**

SQL> select replace('hello world', 'world', 'oracle') from dual;

**OUTPUT**

Results Explain Describe Saved SQL History

| REPLACE('HELLOWORLD','WORLD','ORACLE') |
| --- |
| hello oracle |

1 rows returned in 0.14 seconds CSV Export

**SUBSTR**

SQL> select substr('computer', 1, 4) from dual;

**OUTPUT**

Results Explain Describe Saved SQL History

| SUBSTR('COMPUTER',1,4) |
| --- |
| comp |

1 rows returned in 0.00 seconds CSV Export

**INITCAP**

SQL> select initcap('hello world') from dual;

**OUTPUT**

Results Explain Describe Saved SQL History

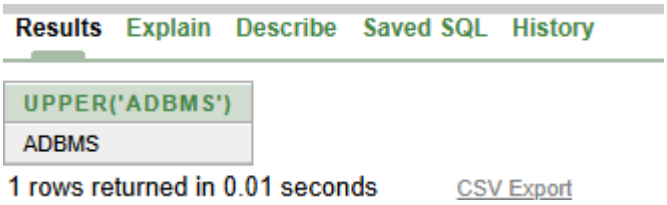
| INITCAP('HELLOWORLD') |
| --- |
| Hello World |

1 rows returned in 0.01 seconds CSV Export

**UPPER**

SQL> select upper('adbms') from dual;

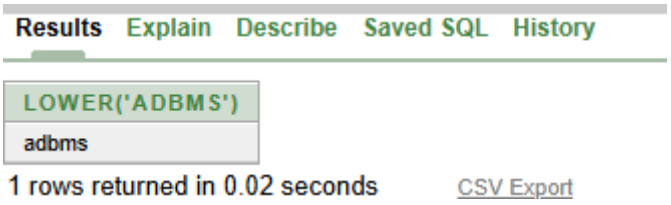
**OUTPUT**




**LOWER**

SQL> select lower('ADBMS') from dual;

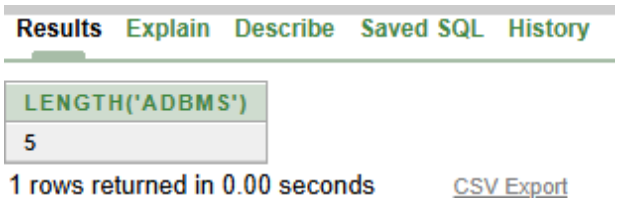
**OUTPUT**




**LENGTH**

SQL> select length('ADBMS') from dual;

**OUTPUT**

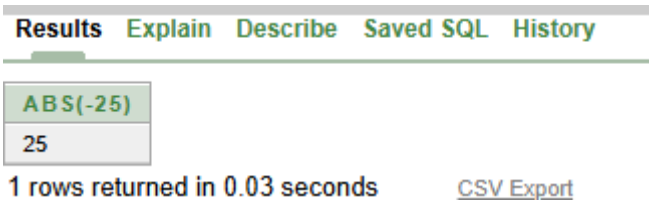


# 11. NUMERIC FUNCTION

**ABSOLUTE**

SQL> select abs(-25) from dual;

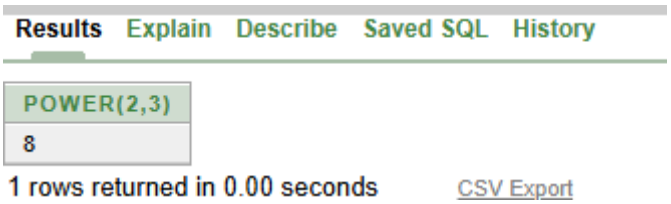
**OUTPUT**




**POWER**

SQL> select power(2, 3) from dual;

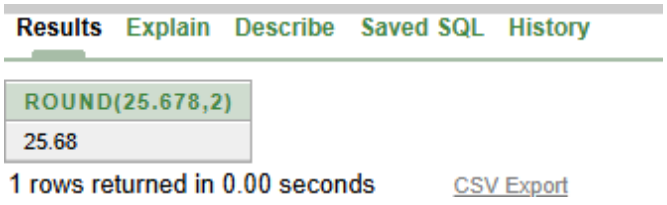
**OUTPUT**




**ROUND**

SQL> select round(25.678, 2) from dual;

**OUTPUT**



# 12. NESTED SUBQUERY

**TABLE 1:**

```
SQL> create table student_sub (student_id number, student_name varchar2(30), dept_id
     number, mark number);
SQL> insert into student_sub values (101, 'savitha', 1, 85);
SQL> insert into student_sub values (102, 'priya', 1, 90);
SQL> insert into student_sub values (103, 'gaja', 2, 75);
SQL> insert into student_sub values (104, 'bavana', 2, 80);
SQL> insert into student_sub values (105, 'raji', 3, 95);
```

**TABLE 2:**

```
SQL> create table department_sub (dept_id number, dept_name varchar2(30));
SQL> insert into department_sub values (1, 'computer science');
SQL> insert into department_sub values (2, 'information technology');
SQL> insert into department_sub values (3, 'artificial intelligence');
```

**IN**

```
SQL> select student_name, mark from student_sub where dept_id in (select dept_id from
     department_sub where dept_name = 'computer science');
```

**OUTPUT**

Results Explain Describe Saved SQL History

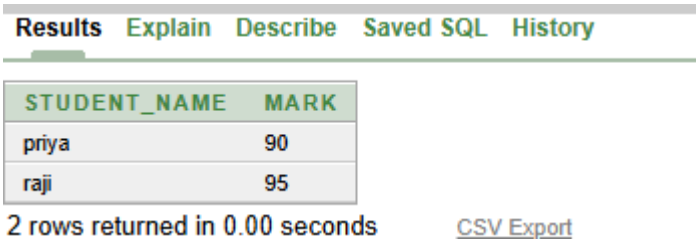
| STUDENT_NAME | MARK |
|---|---|
| priya | 90 |
| savitha | 85 |

2 rows returned in 0.10 seconds CSV Export

**WHERE**

SQL> select student_name, mark from student_sub where mark > (select avg(mark) from student_sub);

**OUTPUT**



Results Explain Describe Saved SQL History

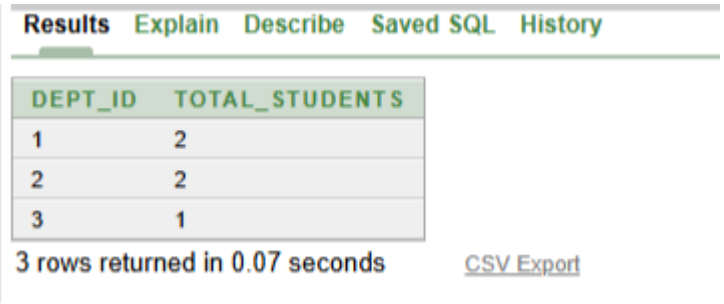
| STUDENT_NAME | MARK |
|---|---|
| priya | 90 |
| raji | 95 |

2 rows returned in 0.00 seconds CSV Export

**GROUP BY**

SQL> select dept_id, count(*) as total_students from student_sub where dept_id in (select dept_id from department_sub) group by dept_id;

**OUTPUT**



Results Explain Describe Saved SQL History

| DEPT_ID | TOTAL_STUDENTS |
|---|---|
| 1 | 2 |
| 2 | 2 |
| 3 | 1 |

3 rows returned in 0.07 seconds CSV Export

**HAVING**

SQL> select dept_id, avg(mark) as average_mark from student_sub group by dept_id having avg(mark) > (select avg(mark)from student_sub);

**OUTPUT**

Results Explain Describe Saved SQL History

| DEPT_ID | AVERAGE_MARK |
|---|---|
| 1 | 87.5 |
| 3 | 95 |

2 rows returned in 0.01 seconds CSV Export

**ORDER BY**

SQL> select student_name, mark from student_sub where mark > (select avg(mark) from student_sub) order by mark desc;

**OUTPUT**

Results Explain Describe Saved SQL History

| STUDENT_NAME | MARK |
|---|---|
| raji | 95 |
| priya | 90 |

2 rows returned in 0.00 seconds CSV Export

**BETWEEN**

SQL> select student_name, mark from student_sub where mark between (select min(mark)from student_sub) and (select avg(mark) from student_sub);

**OUTPUT**

Results Explain Describe Saved SQL History

| STUDENT_NAME | MARK |
|---|---|
| savitha | 85 |
| gaja | 75 |
| bavana | 80 |

3 rows returned in 0.02 seconds CSV Export

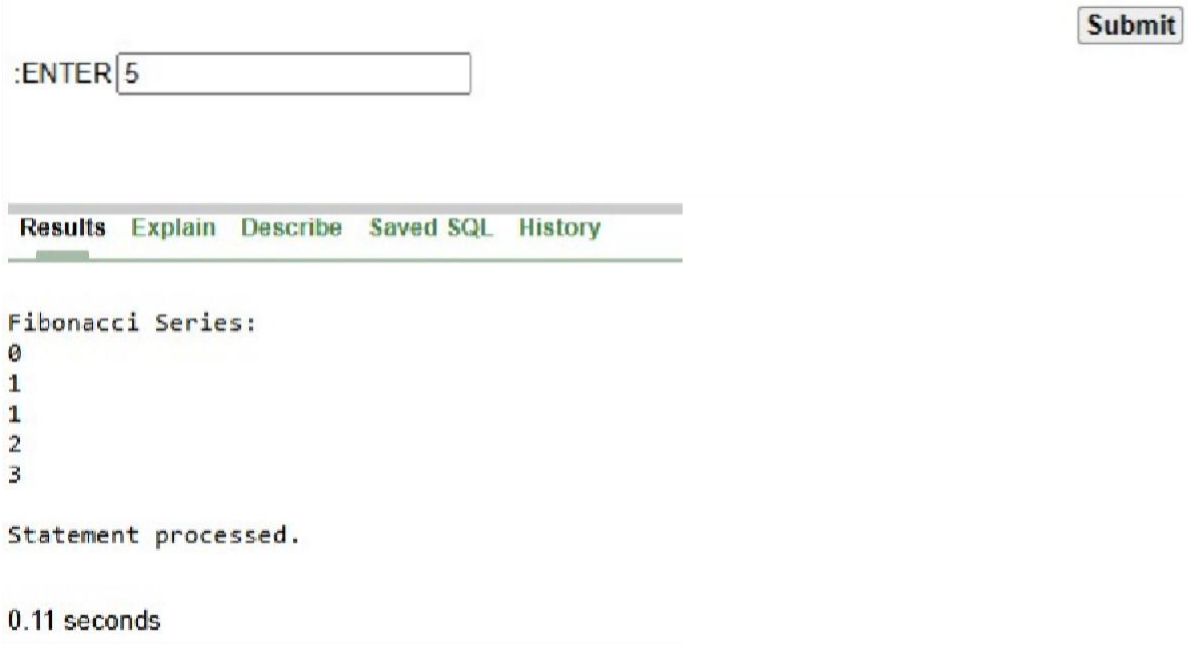
# PL/SQL

## 1. FIBONACCI SERIES

**PROGRAM**

```
declare
   n number := :Enter;
   a number := 0;
   b number := 1;
   c number;
begin
   dbms_output.put_line('fibonacci series:');
   for i in 1..n loop
      dbms_output.put_line(a);
      c := a + b;
      a := b;
      b := c;
   end loop;
end;
```

**OUTPUT**



# 2. FACTORIAL

**PROGRAM**

```
declare
   n number := :Enter;
   fact number := 1;
begin
   for i in 1..n loop
      fact := fact * i;
   end loop;
   dbms_output.put_line('factorial = ' || fact);
end;
```

**OUTPUT**

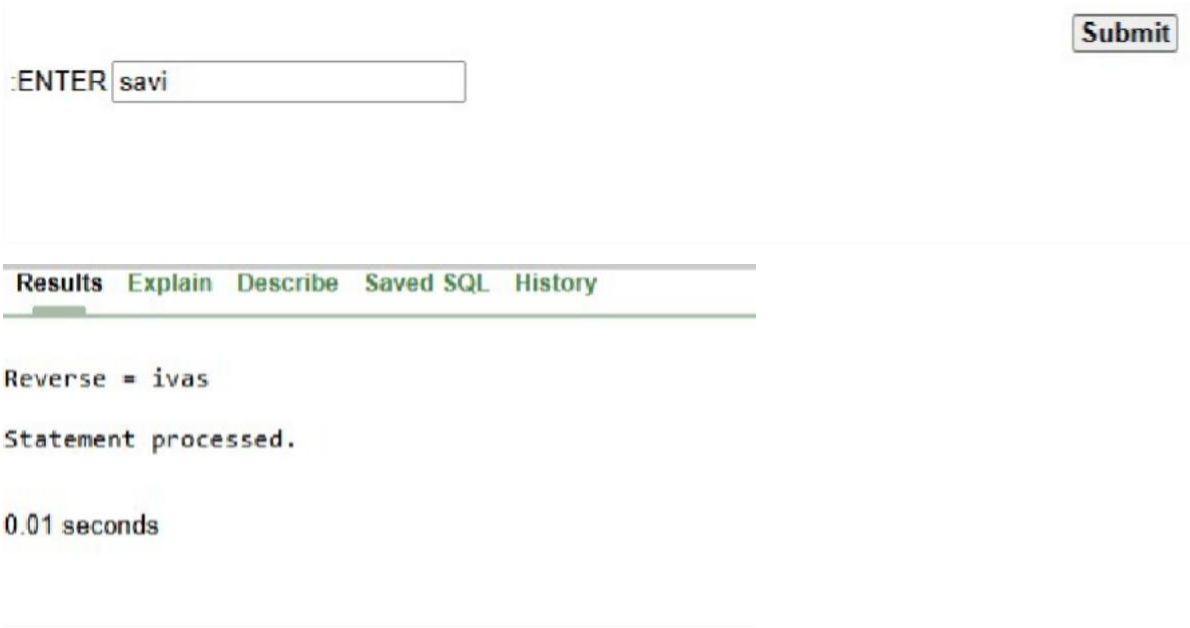


# 3. STRING REVERSE

**PROGRAM**

```
declare
   str varchar(10) := :Enter;
   rev varchar(10) := ' ';
begin
   for i in reverse 1..length(str) loop
      rev := rev || substr(str, i, 1);
   end loop;
   dbms_output.put_line('reverse = ' || rev);
end;
```

**OUTPUT**



# 4. SUM OF SERIES

**PROGRAM**

```
declare
   n number := :Enter;
   total number := 0;
begin
   for i in 1..n loop
      total := total + i;
   end loop;
   dbms_output.put_line('sum = ' || sum);
end;
```

**OUTPUT**



# 5. TRIGGER

**TABLE CREATION**

SQL> create table voter(vid number(10), vname varchar(20), vcity varchar(20));

Table Created

**TABLE INSERTION**

SQL> insert into voter values(1001, 'Savitha', 'pondy');

1 row(s) inserted

SQL> insert into voter values(1002,'Varsha','Villupuram');

1 row(s) inserted

SQL> insert into voter values(1003,'Jessi','Pondi');

1 row(s) inserted

**TABLE VIEW**

Select * from voter

Results Explain Describe Saved SQL History

| VID | VNAME | VCITY |
|---|---|---|
| 1001 | Savitha | pondy |
| 1002 | Varsha | Villupuram |
| 1003 | Jessi | Pondi |

3 rows returned in 0.05 seconds CSV Export

**TRIGGER CREATION**

```
create or replace trigger tud
after update or delete on voter
declare
 msg varchar(20):='trigger fired';
begin
 if updating then
dbms_output.put_line(msg||'record updated');
elsif deleting then
dbms_output.put_line(msg||'record deleted');
end if;
end tud;
```

**UPDATE RECORD**

SQL> update voter set vcity='Pondi' where vid=1002




**DELETION RECORD**

SQL> delete from voter where vid=1003;



# 6. STUDENT MARK AND GRADE CALCULATION

**PROGRAM**

```
declare
   mark number := :Enter_Mark;
   grade varchar(10);
begin
   if mark >= 90 then
      grade := 'a';
   elsif mark >= 80 then
      grade := 'b';
   elsif mark >= 70 then
      grade := 'c';
   elsif mark >= 60 then
      grade := 'd';
   elsif mark >= 50 then
      grade := 'e';
   else
      grade := 'f';
   end if;
   dbms_output.put_line('mark = ' || mark);
   dbms_output.put_line('grade = ' || grade);
end;
```

**OUTPUT**



Results Explain Describe Saved SQL History

```
mark = 85
grade = b

Statement processed.
```

0.02 seconds

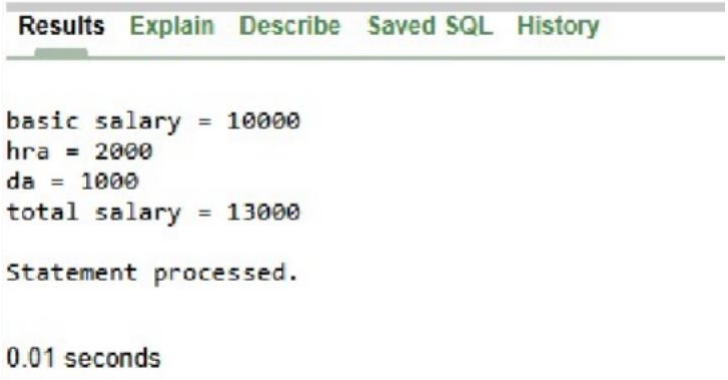
# 7. EMPLOYEE SALARY CALCULATION

**PROGRAM**

```
declare
   basic number := :Enter_Basic;
   hra number;
   da number;
   total number;
begin
   hra := basic * 0.20;
   da := basic * 0.10;
   total := basic + hra + da;
   dbms_output.put_line('basic salary = ' || basic);
   dbms_output.put_line('hra = ' || hra);
   dbms_output.put_line('da = ' || da);
   dbms_output.put_line('total salary = ' || total);
end;
```

**OUTPUT**



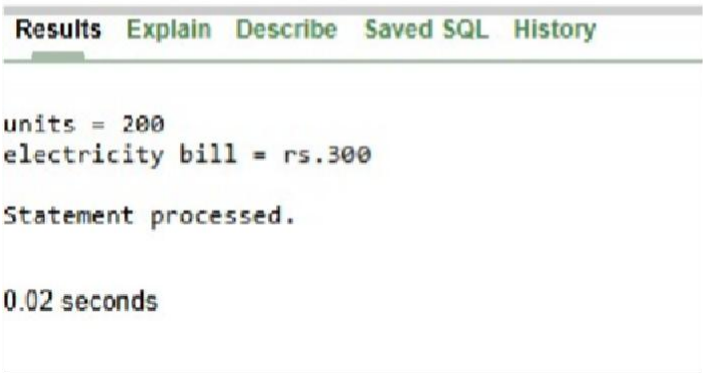
# 8. ELECTRICITY BILL CALCULATION

**PROGRAM**

```
declare
   units number := :Enter_Units;
   bill number;
begin
   if units <= 100 then
      bill := units * 1;
   elsif units <= 200 then
      bill := 100 + (units - 100) * 2;
   elsif units <= 500 then
      bill := 300 + (units - 200) * 3;
   else
      bill := 1200 + (units - 500) * 5;
   end if;
   dbms_output.put_line('units = ' || units);
   dbms_output.put_line('electricity bill = rs.' || bill);
end;
```

**OUTPUT**



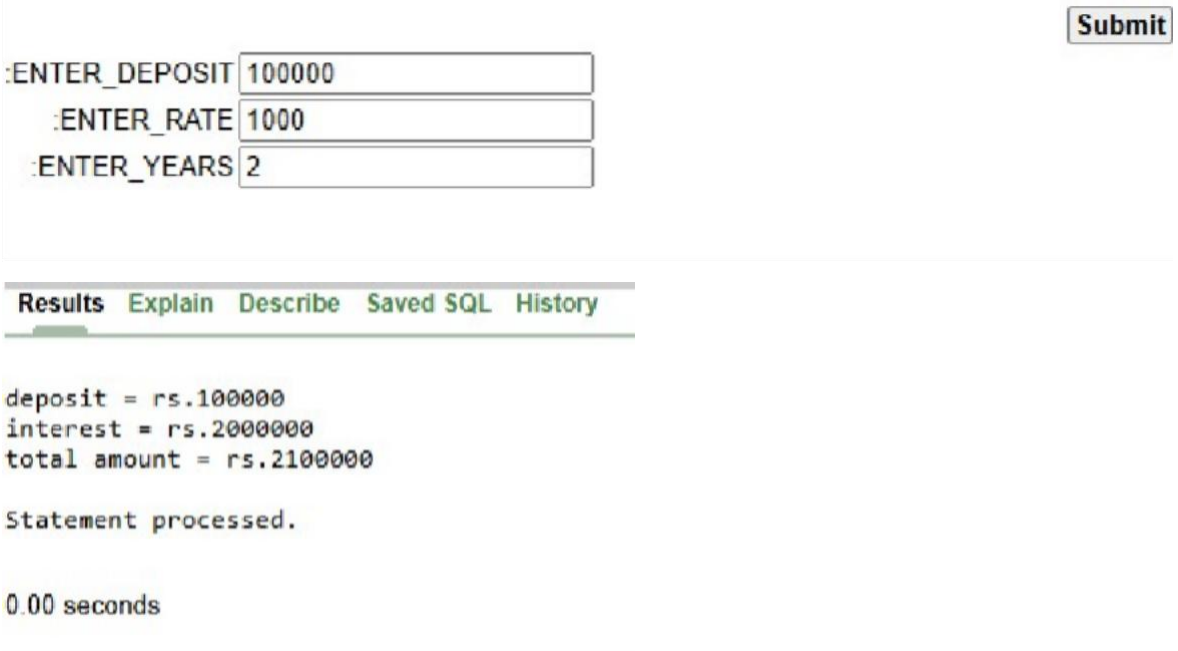



# 9. BANK DEPOSIT AND INTEREST CALCULATION

**PROGRAM**

```
declare
   deposit number := :Enter_Deposit;
   rate number := :Enter_Rate;
   years number := :Enter_Years;
   interest number;
   amount number;
begin
   interest := (deposit * rate * years) / 100;
   amount := deposit + interest;
   dbms_output.put_line('deposit = rs.' || deposit);
   dbms_output.put_line('interest = rs.' || interest);
   dbms_output.put_line('total amount = rs.' || amount);
end;
```

**OUTPUT**

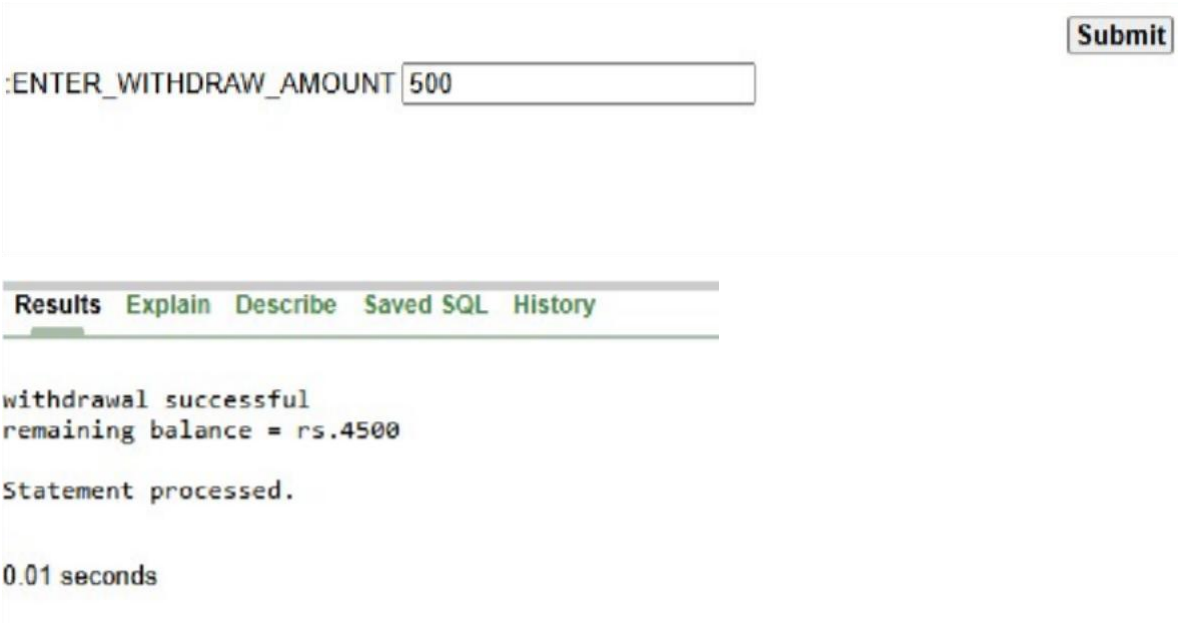


# 10. ATM WITHDRAWAL

**PROGRAM**

```
declare
   balance number := 5000;
   withdraw number := :Enter_Withdraw;
begin
   if withdraw <= 0 then
      dbms_output.put_line('invalid amount');
   elsif withdraw > balance then
      dbms_output.put_line('insufficient balance');
   else
      balance := balance - withdraw;
      dbms_output.put_line('withdrawal successful');
      dbms_output.put_line('remaining balance = rs.' || balance);
   end if;
end;
```

**OUTPUT**



# 11. PRODUCT DISCOUNT CALCULATION

**PROGRAM**

```
declare
   price number := :Enter_Price;
   discount number;
   final_price number;
begin
   if price >= 5000 then
      discount := price * 0.20;
   elsif price >= 3000 then
      discount := price * 0.15;
   elsif price >= 1000 then
      discount := price * 0.10;
   else
      discount := 0;
   end if;
   final_price := price - discount;
   dbms_output.put_line('price = rs.' || price);
   dbms_output.put_line('discount = rs.' || discount);
   dbms_output.put_line('final price = rs.' || final_price);
end;
```

**OUTPUT**



Results Explain Describe Saved SQL History

```
price = rs.5000
discount = rs.1000
final price = rs.4000

Statement processed.
```

0.02 seconds

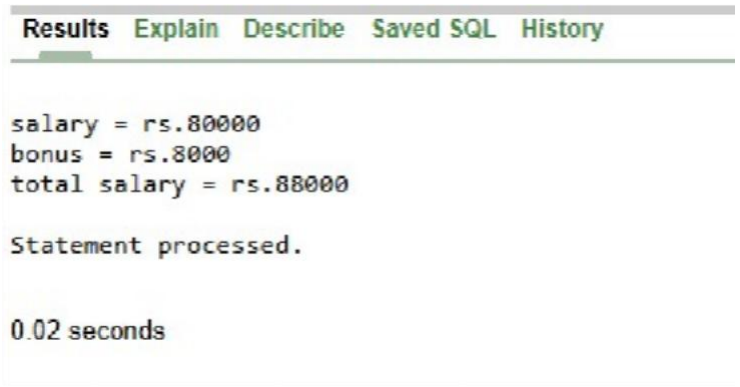
# 12. EMPLOYEE BONUS CALCULATION

## PROGRAM

```
declare
   salary number := :Enter_Salary;
   bonus number;
   total number;
begin
   if salary >= 50000 then
      bonus := salary * 0.10;
   else
      bonus := salary * 0.05;
   end if;
   total := salary + bonus;
   dbms_output.put_line('salary = rs.' || salary);
   dbms_output.put_line('bonus = rs.' || bonus);
   dbms_output.put_line('total salary = rs.' || total);
end;
```

## OUTPUT



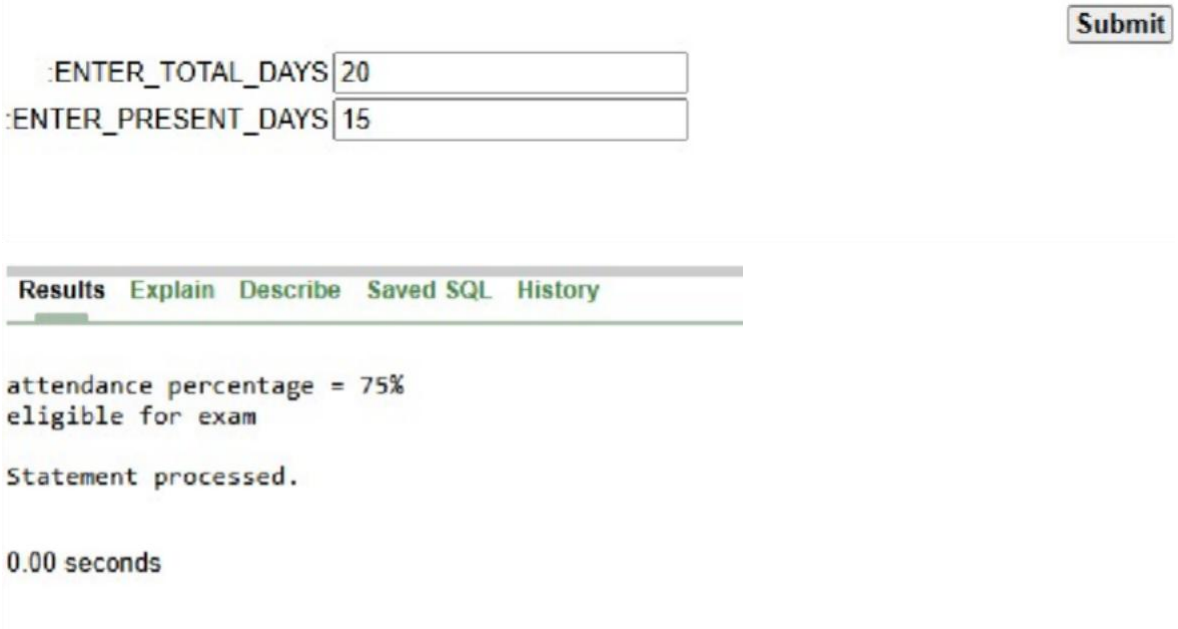



# 13. STUDENT ATTENDANCE ELIGIBILITY

## PROGRAM

```
declare
   total_days number := :Enter_Total_days;
   present_days number := :Enter_Present_days;
   percentage number;
begin
   percentage := (present_days / total_days) * 100;
   dbms_output.put_line('attendance percentage = ' || percentage || '%');
   if percentage >= 75 then
      dbms_output.put_line('eligible for exam');
   else
      dbms_output.put_line('not eligible for exam');
   end if;
end;
```

## OUTPUT



# 14. ARMSTRONG NUMBER

**PROGRAM**

```
declare
   n number := :ENTER;
   temp number;
   rem number;
   total number := 0;
begin
   temp := n;

   while temp > 0 loop
      rem := mod(temp, 10);
      total := total + rem * rem * rem;
      temp := trunc(temp / 10);
   end loop;

   if total = n then
      dbms_output.put_line('armstrong number');
   else
      dbms_output.put_line('not an armstrong number');
   end if;
end;
```

**OUTPUT**



Results Explain Describe Saved SQL History

```
armstrong number

Statement processed.
```

0.06 seconds

# 15. CURSOR

**TABLE CREATION**

SQL> create table customer (id number, name varchar2(20), address varchar2(30), salary number);

**TABLE INSERTION**

SQL> insert into customer values (1, 'savitha', 'pondy', 20000);

SQL> insert into customer values (2, 'priya', 'villupuram', 25000);

SQL> insert into customer values (3, 'gaja', 'chennai', 30000);

SQL> select * from customer;

Results Explain Describe Saved SQL History

| ID | NAME | ADDRESS | SALARY |
|---|---|---|---|
| 1 | savitha | pondy | 20000 |
| 2 | priya | villupuram | 25000 |
| 3 | gaja | chennai | 30000 |

3 rows returned in 0.08 seconds CSV Export

**IMPLICIT CURSOR**

```
declare
   total_rows number;
begin
   update customer
   set salary = salary + 500;

   if sql%notfound then
      dbms_output.put_line('no customers selected');
   else
      total_rows := sql%rowcount;
      dbms_output.put_line(total_rows || ' customers selected');
   end if;
end;
```

**OUTPUT**




```
3 customers selected

1 row(s) updated.

0.07 seconds
```

SQL> select * from customer;




| ID | NAME | ADDRESS | SALARY |
|---|---|---|---|
| 1 | savitha | pondy | 20500 |
| 2 | priya | villupuram | 25500 |
| 3 | gaja | chennai | 30500 |

3 rows returned in 0.00 seconds CSV Export

**EXPLICIT CURSOR**

```
declare
   c_id customer.id%type;
   c_name customer.name%type;
   c_addr customer.address%type;

   cursor c_customers is
      select id, name, address from customer;
begin
   open c_customers;

   loop
      fetch c_customers into c_id, c_name, c_addr;

      exit when c_customers%notfound;

      dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
```

```
    end loop;

    close c_customers;
end;
```

**OUTPUT**

Results Explain Describe Saved SQL History

```
1 savitha pondy
2 priya villupuram
3 gaja chennai

Statement processed.
```

0.03 seconds

# 16. CHECK EVEN OR ODD NUMBER

**PROGRAM**

```
declare
   n number := :ENTER;
begin
   if mod(n, 2) = 0 then
      dbms_output.put_line('Even Number');
   else
      dbms_output.put_line('Odd Number');
   end if;
end;
```

**OUTPUT**







```
Even Number

Statement processed.


0.00 seconds
```

# 17. PALINDROME CHECKER

**PROGRAM**

```
declare
   n number := :ENTER;
   temp number;
   rev number := 0;
   rem number;
begin
   temp := n;

   while temp > 0 loop
      rem := mod(temp, 10);
      rev := rev * 10 + rem;
      temp := trunc(temp / 10);
   end loop;

   if n = rev then
      dbms_output.put_line('Palindrome');
   else
      dbms_output.put_line('Not Palindrome');
   end if;
end;
```

**OUTPUT**



Results Explain Describe Saved SQL History

```
Palindrome

Statement processed.
```

0.02 seconds

# 18. PRIME NUMBER

**PROGRAM**

```
declare
   n number :=:ENTER;
   count number := 0;
begin
   for i in 1..n loop
      if mod(n, i) = 0 then
         count := count + 1;
      end if;
   end loop;

   if count = 2 then
      dbms_output.put_line('Prime Number');
   else
      dbms_output.put_line('Not a Prime Number');
   end if;
end;
```

**OUTPUT**

Submit

:ENTER 7

Results Explain Describe Saved SQL History

```
Prime Number

Statement processed.

0.00 seconds
```

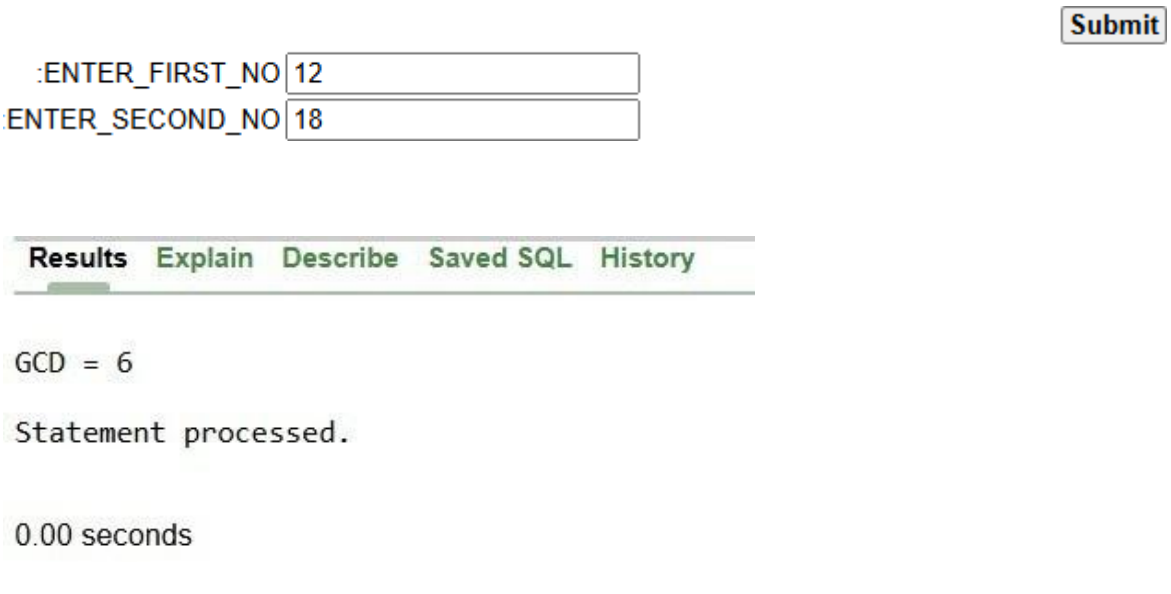
# 19. GCD

**PROGRAM**

```
declare
   a number := :ENTER_FIRST_NO;
   b number := :ENTER_SECOND_NO;
   temp number;
begin
   while b != 0 loop
      temp := mod(a, b);
      a := b;
      b := temp;
   end loop;

   dbms_output.put_line('GCD = ' || a);
end;
```

**OUTPUT**



# 20. OPERATORS

**PROGRAM**

```
declare
   a number := :ENTER_FIRST_NO;
   b number := :ENTER_SECOND_NO;
   choice number := :ENTER_CHOICE_NO;
begin
   if choice = 1 then
      dbms_output.put_line('addition = ' || (a + b));

   elsif choice = 2 then
      dbms_output.put_line('subtraction = ' || (a - b));

   elsif choice = 3 then
      dbms_output.put_line('multiplication = ' || (a * b));

   elsif choice = 4 then
      dbms_output.put_line('division = ' || (a / b));

   elsif choice = 5 then
      dbms_output.put_line('modulus = ' || mod(a, b));

   elsif choice = 6 then
      if a > b then
         dbms_output.put_line('a is greater');
      else
         dbms_output.put_line('b is greater or equal');
      end if;

   elsif choice = 7 then
      if a = b then
```

```
        dbms_output.put_line('both are equal');
    else
        dbms_output.put_line('both are not equal');
    end if;

  else
    dbms_output.put_line('invalid choice');
  end if;
end;
```

**OUTPUT**

